Knowledge Distillation and Deployment of a Command-Parsing Model on a Local Cloud

CS3081 Cloud Computing — Mini Project Technical Report

Aniket Raj · roll no. **230103030** · GitHub `iamankoo` · https://github.com/iamankoo/Cloud-Assignment

Status of this document. This is a *mock demonstration* report for preliminary teacher evaluation, not the graded submission. It describes a complete design and implementation for all seven tasks and the procedure for evaluating each one. **The expensive experiments (teacher labelling, student training, Docker build, cluster deployment, load test, CI runs) have not been executed**; no result in this report was fabricated to fill that gap. Every number carries an evidence tag:

[A] **Actual**: produced by running code in this repository; log in `results/`. [C] **Calculated**: deterministic arithmetic or derivation, reproducible by a script or test. [I] **Illustrative**: hypothetical inputs that demonstrate a method; not a measurement. [R] **Reference**: published by another student (`rahul-singh92/NanoParse`, release v1.0, no licence file); *quoted for context only, not my work and not my result.* [S] **Student-reported**: from my handwritten reflection, no logs exist, unverified. [P] **Planned**: written but not executed.

1 Introduction and architecture

A fixed JSON "frame" with exactly four keys (`intent`, `date`, `time`, `place`) is extracted from a short English command, e.g. *"what is the weather in london"* gives `{"intent":"weather_query", "date":null, "time":null, "place":"london"}`. A 3.8B-parameter teacher (Phi-3.5-mini via Ollama) is far too large to serve on a laptop, so it is used *once, offline* to label 3,000 commands. A 70M-parameter student (EleutherAI Pythia-70M, Apache-2.0) is then fine-tuned to reproduce those labels — *sequence-level knowledge distillation* (Kim and Rush, 2016) — and only the student is deployed behind a FastAPI service on a two-node k3d cluster, with model weights kept in S3-compatible object storage (SeaweedFS), never in the image or in Git.

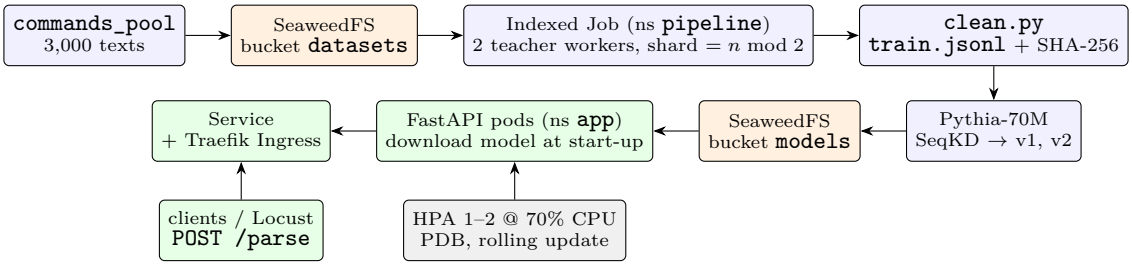


Figure 1: End-to-end architecture. The Ollama server runs on the host and is reached from the cluster at `host.k3d.internal:11434`.

Task	Implementation (repository path)	Evidence level in this report
1	`worker/`, `k8s/worker-job.yaml`, `cluster/`, `prompts/`	sharding and cleaning logic unit-tested on toy records [A]; labelling run [P]
2	`training/`, `tools/predict.py`	config and code reviewed, config completeness tested [A]; training [P]
3	`service/`, `tools/demo_server.py`	29-test suite [A]; stub-model API demo and Locust toolchain smoke test [A]; Docker build [P]
4	`k8s/app.yaml`	manifest fields unit-tested [A]; cluster behaviour [P]
5	`network/`, `scripts/subnet_calc.py`, `tools/nacl_eval.py`	subnet plan and 11/11 packets [A]
6	`.github/workflows/ci.yml`, `Makefile`, `tests/`	stage order and gate logic tested statically [A]; no CI run [P]
7	`tools/locustfile.py`, `scripts/cost.py`, `scripts/slo.py`	cost and SLO arithmetic [C]; worked examples [I]; real load test [P]

Table 1: Per-task implementation and honest evidence level. Environment used: Windows 11, Python 3.10.11; the Docker daemon, k3d, Ollama and `transformers` were not available.

2 Task 1 — Object storage and parallel teacher labelling

Design. k3d (`cluster/k3d.yaml`: 1 server + 1 agent, host port 8080 to 80) hosts SeaweedFS in namespace `data` (single replica, S3 gateway :8333, 2 GiB PVC, 1.5 GiB memory limit), with buckets `datasets` and `models` (`scripts/upload.py`). The teacher prompt (`prompts/teacher_prompt.txt`) states the four-key schema, the eight intents and the three slot rules (copy verbatim, shortest phrase, otherwise null). Ollama is called with `temperature=0`,

`seed=42`; the reply is stored *exactly as generated* in the field `teacher_raw`.

Sharding. An Indexed Job (`completions=parallelism=2`, namespace `pipeline`) starts two pods whose `JOB_COMPLETION_INDEX` serves as `SHARD_INDEX`. Worker i takes the records whose 0-based line number n satisfies $n \bmod K = i$ (K=`SHARD_COUNT`=2) and writes `datasets/teacher/shard-`i`.jsonl`. Each n has exactly one residue, so shards are *complete and disjoint* (no gap, no repeat); for 3,000 records they are *balanced* (1,500 each) [C]; all three properties are asserted by a unit test on 3,000 synthetic ids [A]. Workers are resumable (finished ids are skipped), so a restarted pod does not duplicate work. Caveat: both workers share one host CPU for Ollama, so wall-clock speed-up may be well below 2× (`OLLAMA_NUM_PARALLEL=2` allows two concurrent requests).

Cleaning (`worker/clean.py`). Records are read from all shards, sorted by id, and rejected at the *first* failing rule: (a) not valid JSON; (b) keys not exactly the four; (c) intent not one of the eight; (d) a non-null slot that is not a string occurring in the command (case-insensitive); then (e) duplicate *command text* (the smallest id is kept). The rest is written to `datasets/v1/train.jsonl` with `train.sha256`. Because each record is counted once, $N_{\text{in}} = N_a + N_b + N_c + N_d + N_{\text{dup}} + N_{\text{kept}}$ always holds (asserted in tests [A]). The unit test uses six hand-written toy records [I]:

	input	(a)	(b)	(c)	(d)	duplicates	kept
Toy example [I] [A]	6	1	1	1	1	1	1
Reference run [R]	3,000	1	57	610 (invalid slots)		40	2,292
This project, real run	needs the labelled data of a real run [P]: `make label`, then `make clean-data`; counts go to `results/clean_counts.json`						

The reference row satisfies the identity ($1 + 57 + 610 + 40 = 708$, $3000 - 708 = 2292$) [C], but it is another student's measurement. **Marking checks to reproduce:** two Job pods with different `SHARD_INDEX`; every id in exactly one shard; re-running `clean.py` reproduces the SHA-256 in `train.sha256` (output is written with LF line endings and sorted ids to make this deterministic; tested [A]).

3 Task 2 — Student training by sequence-level distillation

Method. Each example is the line `Command: <text>` followed by the line `JSON: <teacher JSON>`; the *same prompt string* is used by `train.py`, `predict.py` and the service (checked by a test [A]). The loss is token-level cross-entropy on the teacher's best sequence y^* (prompt tokens masked with -100):

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log p_\theta(y_t^* \mid x, y_{<t}^*) \,.$$

Configuration (`training/config.yaml`, read by `make train`; values are [P]): seed 42 (`random` and `torch`), learning rate 5×10^{-5} (AdamW, weight decay 0.01, gradient clipping 1.0), batch size 8, 3 epochs, max length 128, first 1,500 cleaned records (188 steps per epoch, 564 in total [C]), CPU, library versions pinned (torch 2.4.1, transformers 4.44.2, tokenizers 0.19.1, numpy 1.26.4, PyYAML 6.0.2). `make train SUBSET=50` runs the same code on 50 records as a smoke test. Outputs: `training/loss.csv` (one value per epoch), `model.safetensors` and tokenizer, `model.sha256` and `run_info.json` (actual library versions) under `training/output/<version>/`; `scripts/upload.py` copies the directory to `s3://models/v1`. The second version (v2) differs by one recorded setting (another seed via `-seed`, or an extra epoch) and is the target of the rolling update in Task 4.

Evaluation protocol. The instructor's `score.py` is run on `validation.jsonl` (100 commands, never trained on) for three raw-output files: the teacher (same prompt and options as Task 1), the untrained Pythia-70M, and the student. Reported: JSON validity, intent/date/time/place accuracy and full-frame. Expected qualitative outcome [P]: the untrained model scores near 0% validity; the student improves sharply but cannot exceed the teacher's slot accuracy, because it imitates the teacher's *errors* too. For orientation only, another student's published run [R] reports teacher / student v1 / student v2 JSON validity of 94% / 90% / 89% and full-frame accuracy of 44% / 41% / 42%; those numbers are not mine and do not predict my result. The assignment's four-record scoring example (V001–V004) yields validity 75, intent 50, date 50, time 75, place 75 and full-frame 25 [C] (derived by hand from the specification).

Sequence-level vs. logit-based distillation. *Sequence-level* distillation (this project) trains the student on the teacher's *generated outputs* as hard targets: the student sees one decoded string per input. *Logit-based* distillation (e.g. the TRL distillation trainer) trains the student to match the teacher's per-token probability distribution (KL divergence), which also transfers information about near-miss tokens. It is not possible here because the project fixes an Ollama-served Phi-3.5, which returns text rather than logits, and because teacher and student have different tokenizers, so the distributions cannot be aligned token by token. **Limitation of my approach:**

greedy single-sample labels freeze the teacher's mistakes into the dataset, and with only 1,500 examples and 70M parameters the student has no mechanism that guarantees valid JSON (no constrained decoding), so validity must be gated (Task 6).

4 Task 3 — Inference service and container

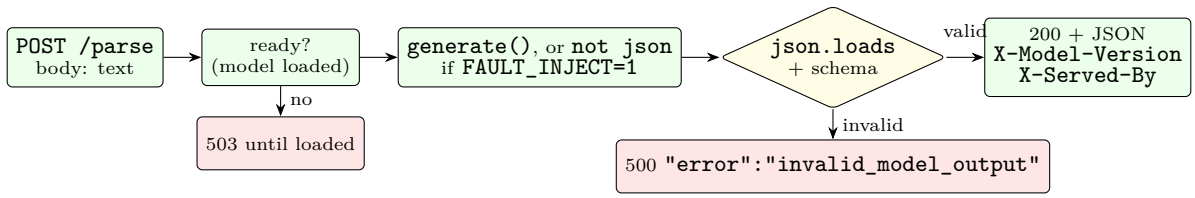


Figure 2: Request handling in `service/app.py`. Broken JSON is never returned with HTTP 200.

API. Port 8000. `GET /healthz` is 200 while the process lives; `GET /readyz` is 503 until the background loader has downloaded the model from `MODEL_URI` on `S3_ENDPOINT` and loaded it (a loader failure is logged and leaves the pod not-ready); `POST /parse` validates every output against `schema.json` and sets `X-Model-Version` (from `MODEL_VERSION`) and `X-Served-By` (the pod name). `MAX_NEW_TOKENS` bounds generation; credentials come only from a Kubernetes Secret.

Container. `python:3.11-slim`, CPU-only `torch==2.4.1` from the PyTorch CPU index (the default wheels pull GPU libraries), every other dependency pinned, no weights in the image, non-root user (uid 10001), and a `.dockerignore` that keeps data, models and results out of the build context. Size limit: 2 GB. The fp32 weights alone are about $70.4\text{M} \times 4\,\text{B} \approx 0.28\,\text{GB}$ [C], which is why they live in object storage and are fetched at start-up.

What was actually run. The 29-test suite (API success and headers, bad intent, extra key, `FAULT_INJECT`, health and ready, cleaning, sharding, manifests, CI order, cost, SLO) passes [A] (`results/pytest_local.txt`). `tools/demo_server.py` runs the *real* `app.py` with a keyword-rule **stub** instead of Pythia-70M: `/healthz` returned 200, `/readyz` 200, a valid `/parse` carried both headers, and `FAULT_INJECT=1` gave HTTP 500 with the exact error body [A] (`results/demo_api_stub.txt`). A 20 s Locust run (4 users) against the stub completed 209 requests with 0 failures at 11.0 req/s [A]. This only proves that the Locust file, header counting and CSV output work; it says nothing about the real service **(stub, not a performance result)**.

Still to run [P]: `docker build -f service/Dockerfile -t command-parser:v1 .`, `docker image inspect command-parser:v1` (size at most 2 GB), and 50 sequential requests under `docker run -cpus=1 -memory=1g`, sampling `docker stats -no-stream` and checking `State.OOMKilled` is false. For orientation, another student's recorded image was about 1.66 GB with a peak of about 737 MiB [R].

5 Task 4 — Kubernetes deployment, scaling and reliability

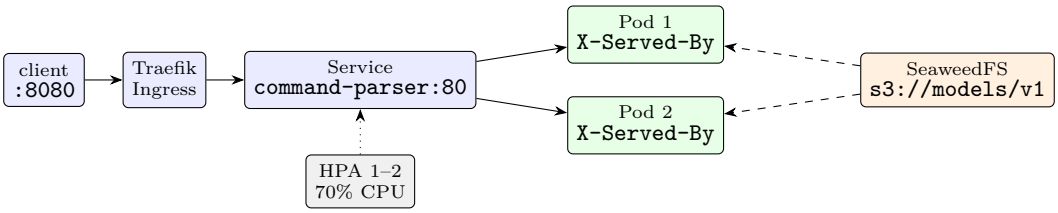


Figure 3: Namespace `app`. Dashed: model download at start-up. Pods become Ready only after the download, so the Service has no ready endpoints while the model loads.

HPA arithmetic [C]: desired = $\lceil \text{current} \times \text{utilisation}/\text{target} \rceil$. Under load, 2 replicas at 140% give $\lceil 2{\cdot}140/70 \rceil = 4$, clipped to the maximum 2; at idle, 2 replicas at 20% give $\lceil 2 \cdot 20/70 \rceil = 1$.

Test procedures [P], all run while Locust is generating load with a longer `-run-time`: (i) *load balancing* — with exactly two Ready replicas count `X-Served-By` in `results/loadtest_headers.json`; each replica should serve at least 30% of the successes; (ii) *rolling update* — `kubectl -n app set env deploy/command-parser MODEL_VERSION=v2 MODEL_URI=s3://models/v2`, target at most 0.5% failed requests (supported by `maxUnavailable=0` plus readiness gating), then `kubectl rollout undo` and confirm that `X-Model-Version` is v1 again; (iii) *recovery* — delete one Pod under load; the replica count must return to 2 with at most 2% failures; (iv) *cold start* — delete all Pods, show the download in the logs, and report the median Pod-creation-to-Ready time over three runs; (v) *HPA* — watch `kubectl get hpa -w` scale 1 to 2 under load and back. **Limitation (required statement):** the cluster runs on one laptop, so the two "nodes" share one machine, one disk and one power supply; this demonstrates Kubernetes self-healing, not fault tolerance across machines. For orientation, another student's recorded cold start was 11 s in three runs [R]; my reflection reports 18.0 s [S] (unverified).

Object (`k8s/app.yaml`)	Configuration [A] (fields verified by unit tests)
Deployment	2 replicas; requests 250m / 512 Mi, limits 1 CPU / 1 Gi; liveness `/healthz` (delay 20 s); readiness `/readyz` (period 5 s, threshold 60, about 5 minutes for the download); RollingUpdate with `maxUnavailable=0`, `maxSurge=1`; `terminationGracePeriodSeconds=30`; Secret-backed S3 credentials; `POD_NAME` via the downward API
Service / Ingress	ClusterIP :80 to container port 8000; Traefik Ingress (`ingressClassName: traefik`) routes `/`, hence `/parse`, to the Service
HPA (autoscaling/v2)	min 1, max 2, CPU utilisation 70%; scale-down stabilisation 60 s (default 300 s) so that scale-down is observable in a short demo
PodDisruptionBudget	`minAvailable: 1`: a node drain can never remove the last ready replica

Table 2: Application manifest.

6 Task 5 — Subnetting and network access control

Allocation (largest first, contiguous from 10.20.0.0, smallest prefix p with $2^{32-p} - 2 \geq h$), reproduced by `scripts/subnet_calc.py` and an independent calculation [A] [C]:

Subnet	hosts h	Network/prefix	Usable range	Broadcast	Usable	Why this prefix
app	1000	10.20.0.0/22	10.20.0.1–10.20.3.254	10.20.3.255	1022	/23 gives 510
public	250	10.20.4.0/24	10.20.4.1–10.20.4.254	10.20.4.255	254	/25 gives 126
data	100	10.20.5.0/25	10.20.5.1–10.20.5.126	10.20.5.127	126	/26 gives 62
management	20	10.20.5.128/27	10.20.5.129–10.20.5.158	10.20.5.159	30	/28 gives 14

Worked example: $\log_2(1000 + 2) = 9.97$, so 10 host bits are needed, giving $p = 22$ and $2^{10} - 2 = 1022$ usable hosts.

NACL for the public subnet (`network/nacl.json`; lowest rule number first, first match wins, default deny):

Dir.	Rule	Protocol/ports	Peer CIDR	Action	Purpose
in	100	all	198.51.100.0/24	deny	abusive range; must precede 110/120
in	110, 120	tcp 80, 443	0.0.0.0/0	allow	web traffic
in	130	tcp 22	10.20.5.128/27	allow	SSH from management only
in	140	tcp 1024–65535	10.20.0.0/22	allow	replies from the app subnet
out	100	tcp 8000	10.20.0.0/22	allow	load balancer to app pods
out	110	tcp 1024–65535	0.0.0.0/0	allow	replies to clients (ephemeral ports)

Why replies need their own rules. A NACL is *stateless*: it does not remember connections, so a reply is a new packet judged on its own and needs an explicit allow on the ephemeral port range; a stateful security group would allow it automatically. **Why order matters.** Because the first match wins, the deny for 198.51.100.0/24 must have a lower number than the allows for ports 80 and 443; if it were numbered above 110, traffic from that range would already have matched the allow. A side effect worth noting: outbound rule 110 also permits TCP 8000 to *any* address, since 8000 lies in 1024–65535. **Evidence.** My own evaluator `tools/nacl_eval.py` decides all 11 hand-written packets as expected (11/11) [A] (`results/nacl_eval_local.txt`). The instructor's `nacl_sim.py` and its six sample packets are not in the repository and were not run [P].

7 Task 6 — Reproducibility, testing and CI/CD

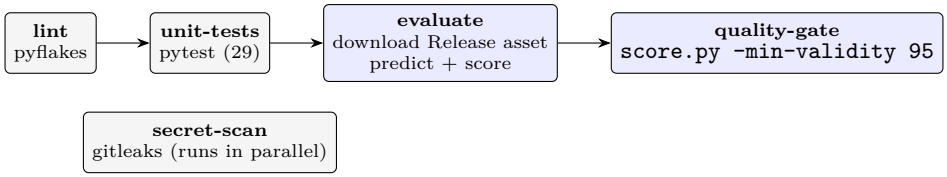


Figure 4: CI stages in the required order. The blue stages are manual (`workflow_dispatch`); they need this project's own model Release and the instructor's `score.py`.

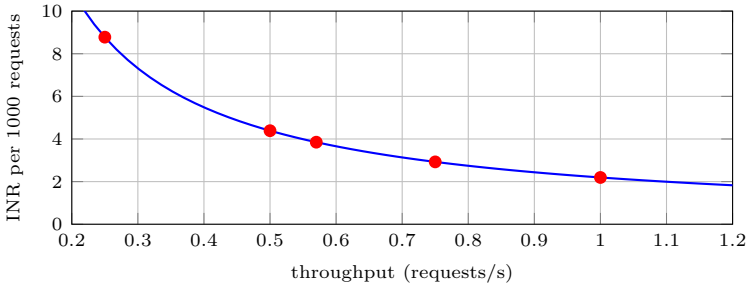
Reproducible build. `make up` runs `scripts/deploy.sh`: create the k3d cluster, namespaces and Secrets from `.env` (never committed; `.env.example` contains development-only values), deploy SeaweedFS, build and import both images, and apply the manifests. Models must already be in `s3://models`. `make down` deletes the cluster. **Tests** (29 passing [A]): service API (5), sharding, cleaning and cleaning CLI, prompt consistency, training-config

completeness, demo server, subnets, NACL, manifest fields, reliability objects, CI stage order, cost and SLO arithmetic. **Quality gate.** `score.py -min-validity 95` exits non-zero below 95% JSON validity. *Showing the gate working* requires a workflow run with the untrained model (`workflow_dispatch`, `model_source=base`) and its URL in the report [P]; no such run exists. Another student's trained model measured 90% validity [R] and would itself fail the gate — a reminder that a 70M model without constrained decoding may need more data, prompt changes or grammar-constrained decoding to reach 95%. **Security.** Secrets live only in Kubernetes and CI secrets; `.gitignore` excludes weights and `.env`; the submission build script scans every file for common key patterns (0 matches [A]). That scan is not gitleaks, which has not been run [P]. Known limitation: one shared S3 identity for all components.

8 Task 7 — Performance, cost, SLO and review

Procedure [P]. Standard run: `locust -f tools/locustfile.py -headless -users 4 -spawn-rate 4 -run-time 2m -host http://localhost:8080 -csv results/loadtest`. Report p50 and p95 (`loadtest_stats.csv`), throughput (req/s), error rate (failures divided by requests; a failure is anything but HTTP 200 within 30 s), per-Pod CPU and memory from `kubectl top pods` sampled during the run, and the cold-start median from Task 4. Targets: error rate at most 1%, p95 at most 5,000 ms.

Cost method. $C_h = r\,(c\,P_{\text{cpu}} + m\,P_{\text{mem}})$ with r replicas billed at their CPU and memory *limits*, and cost per 1000 req $= C_h/(3600 \cdot \text{rps}) \times 1000$. With the exercise prices (3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; hypothetical, as quoted for the exercise rate card by a reference report and consistent with my reflection's 7.90 INR/h) and 2 replicas of 1 vCPU and 1 GiB: $C_h = 2(3.50+0.45) = 7.90$ INR/h [C]. Figure 5 shows how the answer depends on throughput; **the throughput values are hypothetical, so every cost-per-1000 value below is a worked example [I], not my service's cost.**



rps	req/h	INR/1000
0.25	900	8.78
0.50	1800	4.39
0.57	2052	3.85
0.75	2700	2.93
1.00	3600	2.19

Figure 5: Cost per 1000 requests $= 7.90/(3600\,\text{rps}) \times 1000$ [C] [I]; values from `results/cost_slo_worked_examples.txt`.

SLO and error budget. SLO: *"99% of requests succeed within one load-test run"* (window = one 2-minute run); error budget $= (1 - 0.99)N$. Worked examples [I] (request counts hypothetical): for $N = 1000$ the budget is 10 requests; 7 failures use 70% of it (met) and 25 failures use 250% (not met). For $N = 254$ the budget is 2.54 requests, so 3 failures use 118% (not met) [C].

Review table. Each evidence cell holds a number with its evidence tag; where no measurement of the deployed service exists yet, the best available actual or calculated figure is given and the row says so.

9 Student-reported figures (unverified)

The following come from my handwritten reflection (`reflection/230103030.pdf`). **No logs, CSV or command output exist in this project for them, and none were created.** They are shown as reported [S] and must not be read as measurements.

Item	Reported	Status
Pod recovery under load	18.93 s	reported, unverified
Dropped connections during recovery	zero	reported, unverified
Traffic split across replicas	53.4% / 46.6%	reported, unverified (both at least 30%)
Cold start	18.0 s	reported, unverified
Cost (2 replicas) and throughput	7.90 INR/h; 2,052 req/h	7.90 reproduced by calculation [C]
Cost per 1,000 requests	3.95 INR	**arithmetic mismatch**: $7.90/2052 \times 1000 = 3.85$ [C]

3.95 would correspond to 2,000 req/h or to the hourly cost of one replica (7.90/2) (`results/cost_arithmetic_check.txt`);

Concern	Evidence (number, type)	Limitation found	Improvement
Data and storage	3,000 commands split 1,500 + 1,500 [C]; SeaweedFS 2 GiB PVC, 1.5 GiB limit [A] (manifest)	single replica, ReadWriteOnce PVC; an OOM during uploads is reported [S]	replicated volume; multipart uploads
Model quality	no score of my model exists; reference student: 90% validity, 41% full-frame [R]	70M model without constrained decoding; the 95% gate is not guaranteed	more and cleaner data; JSON-grammar decoding; compare v2
Latency and throughput	stub service: 209 req in 20 s, 11.0 req/s, p95 16 ms [A] (framework overhead only); reference service p95 16 s [R]	the stub excludes inference, so no service latency exists	run the standard test; profile `generate()`; cap `MAX_NEW_TOKENS`
Resource utilisation	requests 250m/512 Mi, limits 1 CPU/1 GiB [A]; weights about 0.28 GB [C]; reference peak 556 MiB per pod [R]	inference is CPU-bound, so the HPA reacts only near saturation	tune limits from `kubectl top`; quantise
Reliability and recovery	2 replicas, `maxUnavailable=0`, PDB `minAvailable=1` [A]; reported recovery 18.93 s [S]	single laptop: no node-failure tolerance	multi-node cluster; pre-hydrated model volume to cut cold start
Security and cost	0 secret-pattern matches in the ZIP scan [A]; 7.90 INR/h for 2 replicas [C]	shared S3 identity; no TLS; stateless NACL only	per-service credentials, TLS, network policies

Table 3: Six-row review.

the discrepancy is flagged for correction rather than silently changed. The reflection also proposes pre-hydrated persistent volumes (a shared ReadWriteMany volume, safetensors `mmap`) to cut cold start, and Cilium eBPF network policies in place of stateless NACLs; both are proposals, not tested changes.

10 Reference material, provenance and AI use

The structure of the public NanoParse report (`rahul-singh92/NanoParse`; overview, data and evaluation, deployment, networking, CI, load test, review) was used as a *guide to what a complete report covers*. Its text, code and numbers are not mine; its figures appear here only as clearly tagged [R] context. Its release v1.0 contained two Pythia-70M archives. I once re-hosted them in this repository's Release and later **removed** them because redistribution permission could not be established (no licence file, no written permission). The *original* checksum manifests are retained for attribution only: pythia-70m-v1.tar.gz `3b098817...febab47bfe`, pythia-70m-v2.tar.gz `47ce600a...14b01117` [R] (full values in `docs/RELEASE_METADATA.md`). No model archive generated by this project exists yet. Claude Code (Anthropic) assisted with code, tests, documentation and this report; what it did and how I checked it is in `AI_USE.md`.

11 Limitations and next steps

Not done: teacher labelling, any training, image build, cluster deployment and every Task 4 and Task 7 measurement, CI runs and the failing-gate run, the instructor's `score.py`, `nacl_sim.py` and kit data (not available to me), and gitleaks. **Next steps, in order:** obtain the kit; run `make up`, label, clean and train v1 and v2; publish my own archives with SHA-256 values; score three models; run the standard load test; fill `results/`; trigger the failing and the passing quality-gate runs; rebuild the ZIP from the `final` tag. **Reproducing the demonstration without infrastructure:** `make check` (lint, 29 tests, NACL, subnets), `make demo` followed by `curl`, and `make cost RPS=0.57` or `make slo REQUESTS=1000 FAILED=7` (see `docs/SETUP.md`).

Appendix A — Evidence index

File	Class	Content
`results/pytest_local.txt`	[A]	29 tests passed, pyflakes clean
`results/nacl_eval_local.txt`, `results/subnet_calc_local.txt`	[A]	11/11 packets; subnet plan
`results/demo_api_stub.txt`	[A]	real app with stub model: endpoints, headers, `FAULT_INJECT`
`results/stub_smoke_*`	[A]	Locust toolchain smoke test against the stub (not a service result)
`results/cost_slo_worked_examples.txt`	[C] [I]	cost and SLO arithmetic with hypothetical throughput and counts
`results/cost_arithmetic_check.txt`	[C]	check of the reflection's cost figure
`results/REPORTED_FIGURES.md`	[S]	figures from the handwritten reflection

Appendix B — Commands

Goal	Command
Checks without infrastructure	`make check` (runs lint, test, nacl, subnets)
Local API demo (stub)	`make demo`, then `curl -X POST localhost:8000/parse -H 'Content-Type: application/json' -d '{"text":"what is the weather in london"}'`
Full system (needs Docker, k3d)	`cp .env.example .env`, then `make up`
Label, clean, train, evaluate	`make label`, `make clean-data`, `make train SUBSET=50`, `make train`, `make eval`
Load test, cost, SLO	`make load`; `make cost RPS=<measured>`; `make slo REQUESTS=<n> FAILED=<f>`
Teardown	`make down`

References

Kim and Rush, *Sequence-Level Knowledge Distillation*, EMNLP 2016. Hinton et al., *Distilling the Knowledge in a Neural Network*, 2015. Biderman et al., *Pythia*, ICML 2023. Bastianelli et al., *SLURP*, EMNLP 2020 (CC BY 4.0). Kubernetes (Deployments, Probes, HPA, PodDisruptionBudget), k3d, SeaweedFS, FastAPI, Locust, Ollama, AWS VPC Network ACLs and the Google SRE Book (Service Level Objectives) documentation. Reference report: R. S. Jadoun, NanoParse assignment report, https://github.com/rahul-singh92/NanoParse (context only).